LilyTorch: Supplementary Information

Detailed Numerical Formulations for the Advection Schemes, Boundary Conditions, Pressure Solvers, and Implicit Coupling

Andrea Ferrario

This supplement expands the numerical methods that are stated only briefly in the main manuscript. Section and equation numbers carry an "S" prefix; cross-references of the form "§2.x" or "Eq. (x)" without the prefix point to the main text. Symbols follow the main text: h is the (uniform) cell size, Δt the time step, $\mathbf{u}$ the velocity, p the pressure, ρ the density, ν the kinematic viscosity, and $d(\mathbf{x})$ the signed-distance function (SDF) of an immersed body (positive in the fluid). All fields live on a Marker-and-Cell (MAC) staggered grid with one ghost cell per side.

S1 Spatial discretisation and ghost cells

The domain $\Omega \subset \mathbb{R}^d$ ($d \in \{2, 3\}$) is stored as arrays of shape $(N_x{+}2) \times (N_y{+}2)\,[\times(N_z{+}2)]$, where the single extra layer on each side is the *ghost* (halo) layer used to impose boundary conditions. Pressure p lives at cell centres; the velocity components live on their own staggered face grids (u on x-faces, v on y-faces, w on z-faces). Throughout this supplement we use the index convention that, along a given dimension, slice `[0]` is the low ghost layer, `[-1]` the high ghost layer, and `[1:-1]` the interior.

The convective operator acts on a one-dimensional family of stencils: for a transported quantity ϕ and a face velocity f, the flux through an interior face uses the three aligned values $(\phi_{\text{upstream}}, \phi_{\text{centre}}, \phi_{\text{downstream}})$, selected by the sign of f. The transverse face velocity needed to advect a staggered momentum component is obtained by averaging the appropriate neighbouring velocities to the flux face; for a cell-centred scalar transported along direction d the d-face already coincides with the MAC location of u_d, so no transverse averaging is needed. These details are what make the scheme functions below pure, dimension-agnostic kernels $\lambda(u, c, d)$ of the three upstream/ centre/downstream samples.

S2 Advection schemes

The convective term is written in flux form, $\nabla\cdot(\mathbf{u} \otimes \phi)$, and discretised by reconstructing a face value ϕ_f from the three aligned cell values $u = \phi_{\text{upstream}}$, $c = \phi_{\text{centre}}$, $d = \phi_{\text{downstream}}$ (relative to the local flow direction). Five of the six schemes return a single face value $\phi_f = \lambda(u, c, d)$ and differ only in the limiter λ; the sixth (semi-Lagrangian) replaces the flux form entirely. The advection–diffusion sub-step is advanced with forward Euler and embedded in the outer Heun (RK2) predictor–corrector of the main text (§2.2).

Several limiters are conveniently expressed through the upwind ratio of consecutive gradients

$$r = \frac{c - u}{d - c}, \tag{S1}$$

evaluated with a small floor on the denominator; where $|d - c|$ underflows, the scheme falls back to the first-order upwind value c. The TVD limiters then take the form

$$\phi_f = c + \tfrac{1}{2}\,(d - c)\,\psi(r), \tag{S2}$$

so the limiter function $\psi(r)$ fully characterises the scheme.

QUICK (third-order upwind). The Quadratic Upstream Interpolation for Convective Kinematics scheme [1] fits a parabola through the three points and is stabilised with a double-median (ULTIMATE-style) limiter:

$$\phi_f^{\mathrm{QUICK}} = \tfrac{1}{6}\,(5c + 2d - u), \tag{S3}$$

$$m_{\mathrm{in}} = \mathrm{median}(10c - 9u,\ c,\ d),$$

$$\phi_f = \mathrm{median}\big(\phi_f^{\mathrm{QUICK}},\ c,\ m_{\mathrm{in}}\big). \tag{S4}$$

The inner median bounds the curvature correction, and the outer median clips the result into the monotone interval, which suppresses the dispersive overshoots of the unlimited parabola while retaining third-order accuracy in smooth regions. This is the default scheme.

ADBQUICKEST (Courant-dependent TVD QUICKEST). A TVD form of Leonard's QUICKEST in which the limiter depends explicitly on the local Courant number $C = |f|\,\Delta t/h$. With r from (S1) the limiter is

$$\psi(r) = \max\Big[0,\ \min\big(2(1-C)\,r,\ \tfrac{(2+C^2-3C)+(1-C^2)\,r}{3(1-C)},\ 2(1-C)\big)\Big], \tag{S5}$$

inserted into (S2). The explicit C-dependence tightens the stability bound at large Courant numbers, so this scheme is the preferred high-order option when Δt approaches the advective CFL limit. The implementation requires the actual per-face Courant number to be consistent with the C used in the limiter (a global $\max|f|\,\Delta t/h$ is used).

CUBISTA (Convergent and Universally Bounded Interpolation Scheme). A second-order, universally bounded high-resolution scheme of Alves, Oliveira & Pinho [2], designed for smooth convergence of iterative solvers. Its limiter is the piecewise-linear envelope

$$\psi(r) = \max\Big[0,\ \min\big(\tfrac{3}{2}\,r,\ \tfrac{3}{4}\,r + \tfrac{1}{4},\ \tfrac{3}{2}\big)\Big]. \tag{S6}$$

van Leer. The classical smooth (differentiable) symmetric limiter [3]

$$\psi(r) = \frac{r + |r|}{1 + |r|}, \tag{S7}$$

again used in (S2). It is second-order and TVD, with a gentler shape than the QUICK family.

Central differences (CDS). The plain non-upwind, non-TVD second-order reconstruction

$$\phi_f = \tfrac{1}{2}\,(c + d). \tag{S8}$$

It is the most accurate in smooth, well-resolved flow but admits dispersive wiggles when the cell Reynolds/Péclet number is large; it is mainly used for verification and for the boundary-face fallback of the limited schemes (the lowest/highest interior faces, where the full three-point stencil is unavailable, default to CDS).

Semi-Lagrangian back-tracing. For very large time steps the unconditionally stable semi-Lagrangian advection of Stam [4] is available. Instead of a flux balance, the departure point $\mathbf{x}_d = \mathbf{x} - \Delta t\,\mathbf{u}(\mathbf{x})$ of each grid node is traced backwards and the field is interpolated there,

$$\phi^*(\mathbf{x}) = \phi^n\big(\mathbf{x} - \Delta t\,\mathbf{u}^n(\mathbf{x})\big), \tag{S9}$$

by trilinear (3-D) / bilinear (2-D) interpolation. This removes the advective CFL restriction entirely at the cost of substantial numerical diffusion, and is intended for fast, low-fidelity previews rather than quantitative runs.

Table S1 summarises the schemes. All five flux-form limiters share the registry used both by the momentum advection and by the volume-of-fluid colour transport of the two-phase model (main text §2.7); they are additionally available as a single fused CUDA flux kernel that accumulates the right-hand side in place (main text §2.10).

Table S1: Convective schemes exposed through the `convection_method` configuration key.

Key	Scheme	Order	Notes
`quick`	QUICK + double median	3	default; ULTIMATE-limited
`abdquickest`	ADBQUICKEST	3	TVD, Courant-dependent
`cubista`	CUBISTA	2	TVD, universally bounded
`van_leer`	van Leer	2	TVD, smooth limiter
`cds`	central differences	2	not TVD; verification / fallback
`semi-lagrangian`	Stam back-tracing	1–2	unconditionally stable, diffusive

S3 Boundary conditions

LilyTorch distinguishes the *velocity* boundary conditions, which act on the ghost layer of each MAC component, from the *Poisson* boundary condition, which is a property of the chosen pressure back-end.

S3.1 Velocity ghost-cell conditions

Each velocity component carries an independent list of per-face condition types and values. The faces are ordered

$$(\text{dim0\_lo},\ \text{dim0\_hi},\ \text{dim1\_lo},\ \text{dim1\_hi},\ [\text{dim2\_lo},\ \text{dim2\_hi}]),$$

i.e. (west, east, south, north[, bottom, top]), giving a list of 4 entries in 2-D and 6 in 3-D per component (`bc_type_u/v/w` and `bc_values_u/v/w` in the configuration; an unspecified tail defaults to Neumann). Three elementary ghost operations cover all supported cases. Writing g for the ghost slice and i for the adjacent interior slice:

- **Neumann (zero-gradient)** — copy the interior value into the ghost,

$$\phi_g = \phi_i, \tag{S10}$$

  giving $\partial\phi/\partial n = 0$ at the face. This is the default open/outflow-like condition.

- **Dirichlet (direct)** — for a velocity component *normal* to its own boundary face the wall value coincides with a grid location, so it is written directly,

$$\phi_g = \phi_{\text{wall}}. \tag{S11}$$

  This sets, e.g., a prescribed inflow normal velocity or a zero through-wall velocity.

- **Dirichlet (reflective / mirror)** — for a component *tangential* to the face the wall value sits half a cell outside the last interior node, so the prescribed value ϕ_{wall} is imposed by linear reflection,

$$\phi_g = 2\,\phi_{\text{wall}} - \phi_i, \tag{S12}$$

  which makes the face-interpolated value equal ϕ_{wall} exactly. With $\phi_{\text{wall}} = 0$ this is the standard no-slip mirror.

The three operations are applied in a fixed, stable order (Neumann, then direct Dirichlet, then reflective Dirichlet) so that a wall-face direct write is already committed before the reflective form reads the up-to-date interior value. On the GPU, when all components are contiguous CUDA tensors of the same floating dtype, the entire set of per-face slice copies is dispatched as a single fused `apply_bcs_2d`/`apply_bcs_3d` CUDA kernel (one launch instead of many small slice assignments); an eager PyTorch loop is the fallback otherwise. A typical channel uses a direct Dirichlet inflow on the west face, Neumann outflow on the east face, and reflective no-slip on the lateral walls; a free-stream / open box uses Neumann on all faces.

S3.2 Pressure (Poisson) boundary conditions

The pressure-projection back-end carries its own boundary treatment, selected by `poisson_bc_type`:

- **Neumann** ($\partial p/\partial n = 0$ on every face) — the homogeneous-Neumann Laplacian is exactly diagonalised by the type-II DCT (Section S4), giving the fastest solve. Because the all-Neumann problem fixes p only up to an additive constant, the pressure datum (null space) must be pinned; this is the natural choice for closed/walled domains.
- **Free** (free-space / unbounded) — the right-hand side is zero-padded and convolved with a regularised free-space Green's function, emulating an infinite domain with no reflected pressure waves at the box edges. This is the appropriate choice for an isolated body in open water, often combined with the sponge layer (main text §2.6) on the velocity field.

The variable-coefficient multigrid/MGCG back-end (used for the BDIM mask, variable density, and two-phase flows) inherits the homogeneous Neumann condition at domain faces while embedding the immersed-body condition through the face coefficients $c_{d\pm}$ rather than through the ghost layer; see Section S4. Note that the velocity and pressure conditions are chosen consistently: a Dirichlet (prescribed-velocity) inflow corresponds to a homogeneous-Neumann pressure face, while a Neumann (zero-gradient) velocity outflow pairs with a pressure datum constraint to keep the discrete projection well-posed.

S4 Pressure-projection / Poisson solvers

After the provisional velocity $\tilde{\mathbf{u}}$ is formed, the incompressibility constraint is enforced by solving a Poisson problem for p and correcting $\mathbf{u}^{n+1} = \tilde{\mathbf{u}} - (w\Delta t/\rho)\,\nabla p$. LilyTorch ships three back-ends; the appropriate one depends on the boundary condition and on whether the operator is constant- or variable-coefficient.

S4.1 Spectral DCT solver (constant-coefficient Neumann)

For homogeneous Neumann boundaries with a constant coefficient the discrete Laplacian is diagonal in the type-II Discrete Cosine Transform [5]. Each 1-D second difference has eigenvalues

$$\lambda_k = \frac{2}{h^2}\Big[\cos\Big(\frac{\pi k}{N}\Big) - 1\Big], \qquad k = 0, \dots, N-1, \tag{S13}$$

and the d-dimensional eigenvalue is the sum over axes. The solve is therefore *forward DCT* $\rightarrow$ *divide by* $\sum_{axes}\lambda$ $\rightarrow$ *inverse DCT*, with the zero-frequency (null-space) mode set to zero to fix the pressure datum. This $\mathcal{O}(N \log N)$ path is the fastest and is used for the majority of single-phase validation benchmarks.

S4.2 Free-space Green's function (unbounded)

For an unbounded domain the right-hand side is zero-padded to $2N$ per axis and convolved (via FFT) with a regularised free-space Green's function, so the periodic wrap-around of the padded FFT does not contaminate the interior. In 2-D the eighth-order algebraic smoothing of Hejlesen et al. [6] is used; in 3-D the Gaussian/erf regularisation

$$G(r) = -\frac{1}{4\pi r}\,\operatorname{erf}\Big(\frac{r}{\sqrt{2}\,\sigma}\Big), \qquad \sigma = h, \tag{S14}$$

removes the $1/r$ singularity at the origin. The discrete Green's kernel is precomputed once and cached on disk, so the per-step cost is two padded FFTs plus a pointwise multiply.

S4.3 Geometric multigrid and MGCG (variable-coefficient)

When the BDIM mask μ_0 or a variable fluid/body density makes the projection genuinely variable-coefficient, the discrete operator along dimension d is

$$\frac{c_{d+}\,p_{i+1} - (c_{d+} + c_{d-})\,p_i + c_{d-}\,p_{i-1}}{h^2}, \tag{S15}$$

where the face coefficients $c_{d\pm}$ are the *harmonic* average of the cell values straddling each MAC face (of μ_0 in the BDIM case, or of $\Delta t/\rho$ in the variable-density / two-phase case). The harmonic average keeps the discrete Laplacian symmetric positive-definite and well-balanced across the 10^3:1 density jump at an air–water interface.

The system is solved by a geometric V-cycle [7]:

- **Smoothers**: weighted Jacobi (default relaxation $\omega = 0.7$) or red/black Gauss–Seidel.
- **Restriction**: full-weighting of the residual to the coarse grid.
- **Prolongation**: bilinear (2-D) / trilinear (3-D) interpolation of the coarse correction.
- **Coarsest level**: a trivial $2 \times 2\,(\times 2)$ direct solve.

For anisotropic production grids the V-cycle alone smooths weakly, so it is optionally wrapped in a conjugate-gradient outer iteration (MGCG), with the V-cycle acting as the preconditioner. This roughly halves the solve time on the strongly anisotropic two-phase grids (main text §2.10). Two further accelerations are available and are bit-compatible with the plain multigrid result: a *recycled-Krylov* (deflated MGCG) variant that reuses approximate near-null-space vectors across time steps to help where the smoother stalls, and, for small grids dominated by kernel-launch overhead, capture of the entire native solve as a single CUDA graph.

S4.4 Variable-density and two-phase coupling

When the body density ρ_b differs from the fluid density ρ_f (MuJoCo coupling) an effective density

$$\rho(\mathbf{x}) = \rho_b + (\rho_f - \rho_b)\,\mu_0(\mathbf{x}) \tag{S16}$$

is assembled and the face coefficients $c = \Delta t/\rho_{\text{face}}$ are formed directly on the MAC faces. The same machinery, with $\rho(\alpha) = \alpha\rho_w + (1-\alpha)\rho_a$ from the volume-of-fluid colour field α, drives the two-phase free-surface model. Because the spectral back-ends assume a constant coefficient, all variable-density and two-phase flows are solved exclusively by the multigrid/MGCG path.

S5 Strong (implicit) fluid–structure coupling

This section documents capability that is present in the solver but only alluded to as future work in the main manuscript (limitations, §3.5). By default the fluid and MuJoCo are advanced in a single *explicit*, loosely-coupled (staggered) pass per time step: the fluid reads the body pose, integrates the hydrodynamic wrench, and writes it back once. For light or force-driven bodies whose mean density approaches that of the fluid, this staggered scheme suffers the well-known *added-mass instability* [8]: the explicit wrench lags the body acceleration it induces, and the coupling diverges regardless of how small Δt is.

LilyTorch optionally closes the coupling implicitly within each time step, in the spirit of partitioned solvers such as preCICE [9]. Let x collect the interface unknowns (the per-link wrench / interface state) passed from the body solver to the fluid, and let $\mathcal{F}(x)$ be the value returned after one fluid→body sweep. A converged time step is a fixed point $x^* = \mathcal{F}(x^*)$, found by sub-iterating

$$\tilde{x}_k = \mathcal{F}(x_k), \qquad r_k = \tilde{x}_k - x_k, \tag{S17}$$

until $\|r_k\|$ falls below a tolerance (or a maximum sweep count is reached). Three accelerators control how x_{k+1} is built from the residual history.

Constant under-relaxation. The classical staggered relaxation

$$x_{k+1} = x_k + \omega\, r_k, \tag{S18}$$

with a fixed factor $\omega \in (0, 1]$. $\omega = 1$ reproduces the plain explicit step; a small ω buys stability but smears the true force and converges slowly. This is the baseline.

Aitken (Irons–Tuck) adaptive relaxation. The scalar relaxation factor is recomputed every sub-iteration from the last two residuals [10],

$$\omega_k = -\,\omega_{k-1}\,\frac{\langle r_{k-1},\, r_k - r_{k-1}\rangle}{\|r_k - r_{k-1}\|^2}, \tag{S19}$$

clamped in magnitude to $\omega_{\max}$ and applied as in (S18). This typically converges several times faster than a hand-tuned constant ω while remaining a scalar one-liner; the converged factor is carried into the next time step as the initial guess.

Interface Quasi-Newton (IQN-ILS). The most robust option is the Interface Quasi-Newton with Inverse Jacobian from a Least-Squares model of Degroote et al. [11]. Differences of successive residuals and interface states are stacked into matrices

$$V_k = [\,\Delta r_k,\, \Delta r_{k-1},\, \dots], \qquad W_k = [\,\Delta \tilde{x}_k,\, \Delta \tilde{x}_{k-1},\, \dots], \tag{S20}$$

and the Newton update $\Delta x = -J^{-1} r_k$ is approximated by solving the small least-squares problem $V_k\,\alpha \approx -r_k$ (via a QR factorisation) and forming $\Delta x = W_k\,\alpha + (-r_k - V_k \alpha)$. Columns may be *reused* from a configurable number of previous time steps (the `reuse` window) to enrich the Jacobian model and cut the number of sweeps.

The scheme and accelerator are selected through a `body.coupling` configuration block (`scheme: explicit | implicit`; `accelerator: iqn-ils | aitken | constant`; `reuse`, `tol`, `max_iter`). In implicit mode the constant force-relaxation low-pass used by the explicit path is bypassed. A practical caveat applies to column reuse: a single diverged quasi-Newton sweep can poison the reuse window and silently degrade the method back to an explicit step, so for delicate free-swimmer cases the Aitken accelerator (or `reuse: 0`) is the safer default. Implicit coupling is intended for light or force-driven bodies where added mass dominates; position-controlled swimmers, whose kinematics are prescribed, are advanced with the cheaper explicit path.

References

[1] B. P. Leonard, "A stable and accurate convective modelling procedure based on quadratic upstream interpolation," *Computer Methods in Applied Mechanics and Engineering*, vol. 19, no. 1, pp. 59–98, 1979.

[2] M. A. Alves, P. J. Oliveira, and F. T. Pinho, "A convergent and universally bounded interpolation scheme for the treatment of advection," *International Journal for Numerical Methods in Fluids*, vol. 41, no. 1, pp. 47–75, 2003.

[3] B. van Leer, "Towards the ultimate conservative difference scheme. V. a second-order sequel to Godunov's method," *Journal of Computational Physics*, vol. 32, no. 1, pp. 101–136, 1979.

[4] J. Stam, "Stable fluids," in *Proc. SIGGRAPH*, 1999, pp. 121–128.

[5] J. Makhoul, "A fast cosine transform in one and two dimensions," *IEEE Transactions on Acoustics, Speech, and Signal Processing*, vol. 28, no. 1, pp. 27–34, 1980.

[6] M. M. Hejlesen, J. T. Rasmussen, P. Chatelain, and J. H. Walther, "A high order solver for the unbounded Poisson equation," *Journal of Computational Physics*, vol. 252, pp. 458–467, 2013.

[7] U. Trottenberg, C. W. Oosterlee, and A. Schüller, *Multigrid*. Academic Press, 2001.

[8] P. Causin, J.-F. Gerbeau, and F. Nobile, "Added-mass effect in the design of partitioned algorithms for fluid–structure problems," *Computer Methods in Applied Mechanics and Engineering*, vol. 194, no. 42–44, pp. 4506–4527, 2005.

[9] H.-J. Bungartz, F. Lindner, B. Gatzhammer, M. Mehl, K. Scheufele, A. Shukaev, and B. Uekermann, "preCICE – a fully parallel library for multi-physics surface coupling," *Computers & Fluids*, vol. 141, pp. 250–258, 2016.

[10] B. M. Irons and R. C. Tuck, "A version of the Aitken accelerator for computer iteration," *International Journal for Numerical Methods in Engineering*, vol. 1, no. 3, pp. 275–277, 1969.

[11] J. Degroote, K.-J. Bathe, and J. Vierendeels, "Performance of a new partitioned procedure versus a monolithic procedure in fluid–structure interaction," *Computers & Structures*, vol. 87, no. 11–12, pp. 793–801, 2009.